PROJECT AND TEAM INFORMATION

Project Title

AlgoVision: Interactive DSA Learning, Visualization and Practice Platform

Student / Team Information

Team Name: *Team DSCPP-III-2026-T218*	*AlgoVision*
Team member 1 (Team Lead) *(Last Name, name: student ID: email, picture):*	*Abhishek* *2510012494* *abhishek.in9900@gmail.com* 
Team member 2 *(Last Name, name: student ID: email, picture):*	*Chaitanya Maithani* *2510011693* *maithanichaitanya21@gmail.com* 
Team member 3 *(Last Name, name: student ID: email, picture):*	*Arihant Rawat* *2510012102* *rawatarihant881@gmail.com* 
Team member 4 *(Last Name, name: student ID: email, picture):*	*Chaitanya Kharkwal* *2510010395* *2510010395@geu.ac.in* 

PROPOSAL DESCRIPTION (10 pts)

Motivation (1 pt)

DSA sits at the heart of coding and problem-solving. Still, a lot of beginners hit a wall when all they do is stare at code and hope it "clicks." It often doesn't.Linked Lists make that painfully obvious. Pointers too. Dynamic memory? Even worse. Nodes connect, disconnect, shuffle around, and the whole structure keeps shifting under your feet. Blink, and you're lost.Sure, a student might memorize the C++ syntax for insertion or deletion. Push. Enqueue. They can type it out fast. But do they actually see what the program is doing inside memory while it runs? Most of the time, no.AlgoVision is our way of fixing that mess.

State of the Art / Current solution (1 pt)

Right now, most students end up juggling a bunch of places just to learn DSA. One site explains the theory decently. Another has nicer animations. Then there's LeetCode—great for grinding problems and getting ready for interviews.Helpful? Sure. Clean and simple? Not really.If you're new, the "what do I do next?" question hits fast. And it hits a lot. You finish reading about a topic, then you bounce to some visualization, then you hunt for practice questions. . . and suddenly you're three tabs deep with no real plan.AlgoVision isn't trying to dethrone LeetCode. It's not another online judge, either. The whole point is to stitch the learning flow together into three straightforward

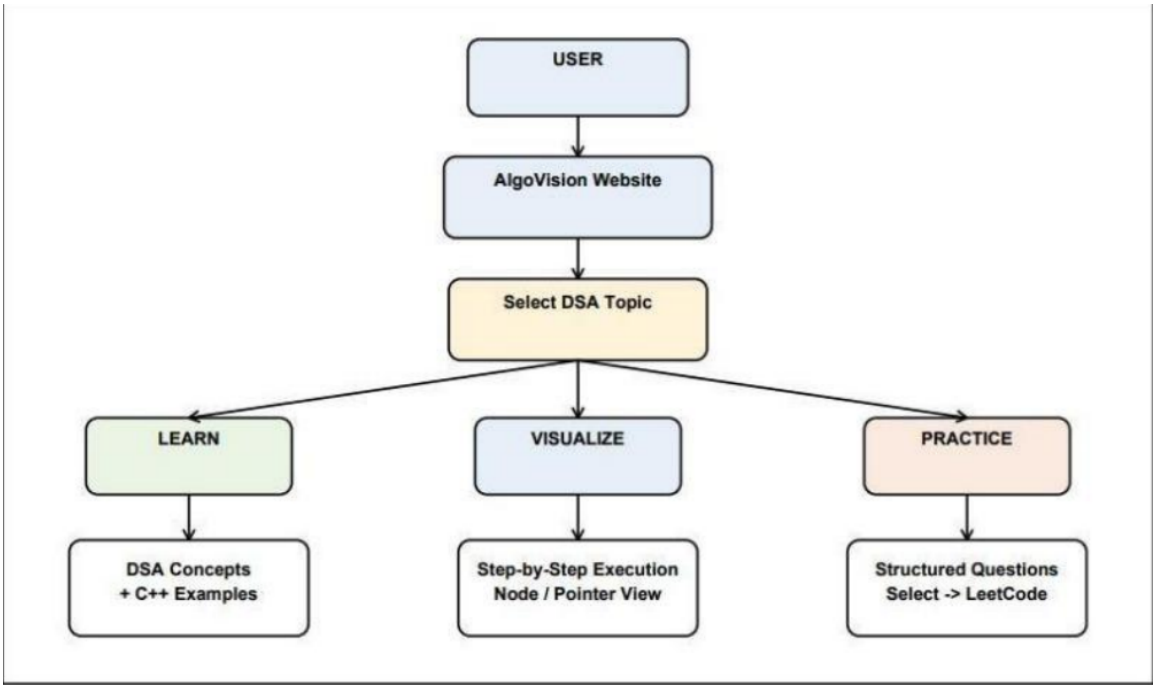
Project Goals and Milestones (2 pts)

- *-Build a simple and responsive DSA learning website using HTML, CSS and JavaScript, with C++ examples.*
- *-Arrange important topics such as Arrays, Strings, Linked Lists, Stacks, Queues, Trees and Graphs in a clear order.*
- *-Create visualizations for Linked Lists, pointers/DMA, Stack operations and Queue operations. - Use nodes, arrows, labels and highlighted states to show what changes during each step.*
- *-Add Previous, Next, Play and Reset controls for the visualizations.*
- *-Organize practice questions by topic and difficulty and open the selected question on LeetCode.*
- *-Add search, filters and simple browser-based progress tracking.*
- *-Complete the work in stages: interface, question data, visualizations, controls, testing and final integration.*

Project Approach (3 pts)

We built this project for anyone using a modern web browser with support for HTML, CSS, and JavaScript. You'll need internet access when opening a LeetCode problem, but you can use the main learning pages and any visuals that get saved locally even if you're offline. All the question links and topic classifications should stay up to date. AlgoVision will show how specific DSA operations work to help with learning. It's not meant to be a full C++ compiler, debugger, or an online judge. Since this is just a semester-long project, we're prioritizing features that actually make sense to visualize. Your progress won't be tied to an online account—everything gets saved right in your own browser.

System Architecture (High Level Diagram)(2 pts)



Project Outcome / Deliverables (1 pts)

The final project will be a working web-based platform for learning, visualizing and practicing DSA. A student will be able to choose a topic, read a short explanation, view C++ examples and use interactive visualizations for the operations we support. The visualization will show changes such as node creation, pointer movement, link updates and Stack or Queue positions through Previous, Next, Play and Reset controls. The Practice section will contain organized questions with search and difficulty/topic filters and will open the selected problem on LeetCode. Basic completion progress will be saved in the browser. The final submission will include the website code, C++ examples, visualization modules, question data, testing and project

Assumptions

We're going to tackle the project one topic at a time. First up: taking DSA topics and breaking them into bite-sized concepts and operations. We'll find good practice questions for each, and make sure to include the title, difficulty level, and a handy LeetCode link. For the visual part, every operation will get split into clear, step-by-step states. JavaScript will handle storing these states and updating the visuals—like nodes, arrows, and labels—right on the webpage. Take Linked List insertion, for example. You'll see the original list, then a new node gets created, we set its value, update pointers, and finally connect everything up. Stack and Queue will follow the same idea. The interface comes together with HTML and CSS. Once we have the main parts working, we'll add features like controls, search, filters, saving your progress locally, and testing.

References

-T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, MIT Press link:-https://mitpress.mit.edu/9780262046305/introduction-to-algorithms/

-M. A. Weiss, Data Structures and Algorithm Analysis in C++, Pearson.

link:-https://www.pearson.com/en-us/subject-catalog/p/data-structures-and-algorithm-analysis-in-c-/P200000003459

-B. Stroustrup, The C++ Programming Language, Addison-Wesley.

Link:- https://www.stroustrup.com/4th.html

-LeetCode - external practice platform used for redirects to selected DSA problems

Link:- https://leetcode.com/https://leetcode.com/

-C++ Standard Library documentation - arrays, strings, vectors, stacks, queues, pointers and Dynamic Memory Allocation.

Link:- https://en.cppreference.com/w/cpp/standard_library